Study Material (class 9)

1. switch case allows the following data types as switch parameter and case labels: byte, short, char, int , **String**
(String is allowed since JAVA 7)
And the following types are **not allowed** : long, float, double, boolean

2. System.out.println(10+20 + "Hello" +10+20);
output: 30Hello1020
** JAVA evaluates from left to right,
10+20 + "Hello" 10+20;
30+"Hello"+10+20
"30Hello"+10+20
"30Hello10"+20
"30Hello1020"
30Hello1020

3. **Finite Loop** – A loop that executes a limited number of times and then stops. Example: A loop running from 1 to 10.

Infinite Loop – A loop that never terminates unless manually stopped or a break condition is met. Example: while(true) { }

Fixed Loop – A loop that runs a predetermined number of times, typically controlled by a counter. Example: for (int i = 0; i < 5; i++)

Null Loop – A loop that runs without performing any operations inside its body. Example: for (int i = 0; i < 1000; i++);

[The term **Null Loop** isn't commonly used in Java programming. A loop with no operations inside is typically referred to as a **loop with an empty body**.]

4. **Entry Controlled Loop** – The loop condition is checked **before** executing the loop body. If false initially, the loop **does not execute** at all. (e.g., for, while)

Exit Controlled Loop – The loop condition is checked **after** executing the loop body. The loop **executes at least once**, even if the condition is false. (e.g., do-while)

5. Jump Statements:
break – Exits a loop or switch statement immediately.
continue – Skips the current iteration and moves to the next loop cycle.
return – Exits from the method and optionally returns a value.

6. System.exit(0) – Terminates the program immediately. The argument 0 indicates normal termination, while non-zero values indicate errors.

7. **Fall through** occurs in a switch statement when a case does not have a break, causing execution to continue into the next case.

```
int num = 2;
switch (num) {
  case 1:
    System.out.println("One");
  case 2:
    System.out.println("Two");  // No break, so falls through to next case
```

```
   case 3:
      System.out.println("Three");
}
```
output:
Two
Three

8. Precedence of operators in JAVA (Order of execution):

Precedence:

1. parenthesis ()
2. Unary operators: ++ , -- , ! (Logical NOT), ^ [Math.pow()]
3. Arithmetic Operators: (i) / % *
   (ii) + -
4. relational: > < >= <= == !=
5. Logical Operators: (i) && (Logical AND)
   (ii) || (Logical OR)
6. Assignment: =

9. Type Cast Operator: ( )

10. Math.random() returns a double value between 0 and 1 [0 is included and 1 is excluded]

11. Data types in order: (1 byte = 8 bits)

Data Type	Size in bits	Size in bytes	Default Value
byte	8	1	0
char	16	2	'\u0000'
short	16	2	0
int	32	4	0
long	64	8	0L (L as suffix indicates a long value)
float	32	4	0.0f /0.0 (f as suffix indicates a float value)
double	64	8	0.0d /0.0 (d as suffix indicates a double value)

*** Every escape sequence (\n, \t, \b etc) takes 2 bytes (they are characters)

* 10 takes 4 bytes in memory (int)
  12.5 takes 8 bytes in memory (double)
  12.5f takes 4 bytes in memory(the suffix f indicates float)
  12.5d takes 8 bytes in memory (the suffix d indicates double)
  'A' takes two bytes in memory (any thing between ' ' is a character and takes 2 bytes or 16 bits)
  "JAVA" takes 8 bytes in memory (4 characters = 4 x 2=8 bytes)

12. Data types in ascending order:
    byte, char, short, int, long, float, double

Conversion from smaller to larger data type is Implicit Type Casting (Coercion)
    Example: double a=10; [int to double]
Larger to smaller is explicit type casting/conversion:
    Example: int a=(int)12.5; [double to int]

13. Extension of JAVA Source Code: **.java**
    Extension of JAVA Byte Code: **.class**

14. Ternary (?:) is also known as conditional Operator.

15. Access Specifiers:
public: Accessible from anywhere (inside or outside the class, package, or project).
private: Accessible only within the class where it is defined.
protected: Accessible within the same package and subclasses (even in different packages).
default: Accessible only within the same package (package-private)

16. Operation between multiple data types results in the largest of them.
Example: int + float + short = float
int+ double + char= double
int + int + int = int

Exception : char+char = int
System.out.println('A' + 'B'); -→ 131

Exception in Exception: char ch= 'A';
ch= ch+2; (char is updated and stored in itself)
System.out.println(ch); → C

17. | x| means absolute of x . which basically returns positive value of x . Written in JAVA using Math.abs(x)

18. Principles of OOP (Object-Oriented Programming) in Java

1. **Encapsulation** – Bundles data (variables) and methods (functions) into a single unit (class) and restricts access to certain components using access modifiers.
2. **Abstraction** – Hides implementation details and shows only essential features, typically through abstract classes or interfaces.
3. **Inheritance** – Allows one class to inherit the properties and methods of another class, promoting code reusability.
4. **Polymorphism** – Enables a single method or operator to work in different ways depending on the context (method overloading and overriding).

19. **JVM**: Converts Java bytecode into machine code to run on any system.
JRE: Provides the environment to run Java applications, including the JVM and libraries.
IDE: A software for writing, testing, and debugging code. (like BlueJ)
JDK: A toolkit for developing Java applications, including the JRE and development tools
20.

Property : **a/b = 0 ∀ (a,b) ∈ Integers and a<b**
It means, dividing a smaller number by a larger number results as 0 if a and b are integers and a<b

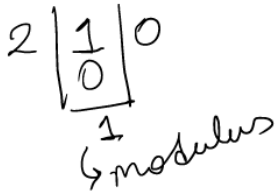
Property: **a%b = a ∀ (a,b) ∈ Integers and a<b**
It means, smaller number modulo larger number results as the smaller number if a and b are integers and a<b

System.out.println(Math.pow(25, 1/2));

its $25^{1/2}$

Computer first calculates ½ which is 0

Now the expression is 25^0 which 1 , since Math.pow() returns as double, the final value is 1.0



You can get the real value by using floating point literals:

System.out.println(Math.pow(25, 1.0/2.0));

its $25^{1.0/2.0}$
Computer first calculates 1.0/2.0 which is 0.5

Now the expression is $25^{0.5}$ which is equivalent to $\sqrt{25}$
And we get 5, since Math.pow() returns double value, we get 5.0

21.

Errors:

Compilation Error: grammatical mistake, type mismatch etc

Example:	int a=10	missing semicolon
Example:	inta= 10;	no space between int and a
Example:	int a=12.5;	type mismatch

Runtime Error: Dividing a number by 0

Example:	int k= a/0;	
Example:	int a= 1/2;	
	int b= c/a;	evaluating c/0 since a is 0
Example:	int p= 4%2;	p is now 0
	int k= b/p;	evaluating b/0

Logical Error: The program executes but gives wrong output:

```
int a= 10+20;
System.out.println("Difference"+ a);
```

Comments:	//comment	single line
	/* comment …….. */	multiline

** In ICSE, compilation errors are also referred to as syntax errors, although syntax errors are a subset of compilation errors